华为「AI 模型工程师 — 多模态大模型」面试速成指南

> **说明**：本文档为对外分享版，前雇主项目的量化指标已做量级化处理，方法论与话术保持原样。
> 用法：第 1 章改简历（半天），第 3 章背代码（两个半天），第 2、5 章通勤/睡前过。 全文约 3 天可闭环。红线在后面，先看不吃亏。

0. 先定调：你的牌面和这个岗位的接口在哪

你不是"零基础转多模态"，你的真实资产是**多模态融合 + 几何对齐 + 大规模训练调优 + 端侧部署**四件事的实操。华为这个岗（2012/诺亚/车 BU/终端都有类似坑位）真正要的是三类人之一：

岗位方向	关键词	你的匹配度
VLM 算法（图文/视频理解、对齐、SFT）	CLIP、Q-Former/MLP projector、LLaVA、数据配比、RLHF/DPO	中：训练调优、数据闭环迁移得过去，但**没训过 LLM**
多模态感知/空间智能（车 BU、具身）	BEV、Occ、深度、点云-图像融合、3D grounding	**高**：这是你的主场
端侧大模型/推理加速	量化、蒸馏、算子、昇腾 CANN/ATC、显存	**高**：ATC→MDC610 落地过

定位话术（开场 30 秒自述就用它）：

> "我做过的是把视觉、激光两种异构模态在统一特征网格上做对齐融合，并负责过从数据生产、指标归因到大模型训练稳定性、再到 ONNX→ATC→MDC 端侧部署的整条链路。多模态大模型里我最有底气的是**跨模态对齐、长序列建模、大规模训练调优和推理落地**这四块，模型侧的 VLM 结构和 SFT/偏好对齐我在系统补齐。"

诚实但不掉价，且直接命中华为最看重的"能落地 + 能调优"。

1. 简历往多模态靠（不改事实，只换词汇）

1.1 关键词映射表（左边是你现在的写法，右边是 HR/技术面想看到的写法）

你现有表述	改成（同义、但要能扛住追问）
视觉支路 + 激光支路 BEV 融合（BevFusionV2、SE、CBAM）	**异构模态特征对齐与门控融合**：视觉 128ch@256×256 与点云 128ch@256×256 在统一 BEV token 网格上对齐，SE 通道门控解决两模态密度/尺度不匹配
ViewTransformer + LSS 深度分布（D=99, sid 模式）	**几何感知的视觉 token 重组**：7 视角 3×384×704 经深度分布加权投影为 65,536 个 BEV query，实现图像 token 到 3D 空间的显式定位（空间智能/VLA 的地基能力）
temporal_history + scene-aligned 8 段时序切分	**长序列时序建模的显存-长度权衡**：按 scene 对齐切段避免跨场景污染，时序历史缓存控制在 X 帧，平衡 mIoU 与显存
多尺度深监督 + ASPP 粗头 w=0.2（loss 降一个量级）	**多尺度深监督抑制收敛不稳**：粗头辅助损失加权 0.2，收敛期 loss 降一个数量级，解决深层任务头梯度稀疏
32×32 混合矩阵替 3-tap（显存降约 8%）	**显存墙下的结构重构**：用 32×32 混合矩阵替代 3-tap 大核，显存 −7.1GB（−8%），保住 batch 与分辨率
OCC GT 数据闭环（数万个场景/百万级关键帧/ray-casting 三态）	**大规模多模态数据生产线与评测基准构建**：ray-casting 区分 free/occupied/unknown 三态，scene 级时序采样，零激光评测协议（万级帧）+ 双 seed 噪声地板
ONNX→ATC/AOE→MDC610 .om；Decode/TopK/NMS 解耦	**端侧推理优化与算子改造**：不支持的算子解耦 + OpenCV 向量化，ROI 超分误检 约 5%→约 1.4%，完成 opset11/16 到昇腾 .om 全链路

你现有表述	改成（同义、但要能扛住追问）
标定三层法（4-DOF 同位变换/PCA 切线拟合/递进降级）	**多模态时空对齐与鲁棒性设计**：scale-invariant 残差 + 状态机退化恢复，角点补全 约 9px→约 3px、失败恢复率 约 87%

1.2 三条 bullet 的改写示例（直接可贴）

改前：负责视觉与激光 BEV 融合模块开发与训练调试。

改后：

> 主导 7 相机 + 4 激光的异构模态融合主干（BevFusionV2 + SE 门控）：两条支路在 0.6m/格、256×256 的 BEV token 网格上对齐，SE 通道重加权缓解模态密度差；8 卡训练集群 训练 12/24 epoch EMA，显存峰值 约 89GB（十万级行 batch点云），通过混合矩阵重构降到 约 82GB 后保住分辨率。

改前：负责 Occupancy 真值生产与评测。

改后：

> 负责 Occ 监督生产线与评测基准（数万个场景 / 百万级关键帧）：ray-casting 区分 free/occupied/unknown 三态，标注口径与转换器版本双向核查（发现 filter 版本反转导致 mask 体积 约 0.64 倍、occ mIoU 降约 3 个点 的隐性 bug）；设计 万级帧评测协议与双 seed 噪声地板，把"模型变好"与"评测抖动"分开。

改前：负责模型在 MDC 平台的部署。

改后：

> 完成感知模型 ONNX(opset11/16)→ATC/AOE→MDC610 .om 的端侧落地：对 NPU 不支持的 Decode/TopK/NMS 做算子解耦 + OpenCV 向量化重写，端侧 ROI 超分误检 约 5%→约 1.4%。

1.3 必须补进简历的"多模态新词"（真实可辩护的最小增量）

只在**你真的动过手/读透过**的前提下加，且要能接住追问：

- **CLIP / InfoNCE 对比学习**：把"视觉-文本对齐"接到你现成的"视觉-激光对齐"经验上（对齐就是对齐，差别在监督信号与 loss）。
- **ViT / patch tokenization**：你在 backbone 里做的就是多尺度特征 token 化，能讲清 patch 划分、位置编码、CLS token、计算复杂度 $O(N^2)$ 与窗口注意力。
- **projector（MLP / Q-Former）**：讲清"冻结视觉塔 + 只训二层 MLP"为什么在 LLaVA 里够用，Q-Former 的 query 压缩为什么在 BLIP-2 里用来降 token 数——**这个你可以用"BEV query 压缩/重排"经验直接类比**，是你最容易讲出深度的点。
- **LoRA / 混合精度 / ZeRO**：你有显存墙实战，讲 LoRA 的 rank/alpha、ZeRO-2/3 的切分对象即可。
- **评测集**：MMBench / MMMU / SEED / POPE（幻觉）、RefCOCO（grounding）各评什么、指标是什么。

1.4 红线（比写得好更重要）

1. **不说自己训过 LLM/VLM**。说"补课中/做过同构的跨模态对齐与大规模训练调优"。
2. **不报参数量**（百万级参数量 这种会被"这么点也叫大模型"打穿）。改说结构设计意图与单变量控制（恒等初始化保证可比）。
3. **不说论文已发表**。
4. **不说做过 SLAM**（只说视觉几何、位姿对齐、ICP、4-DOF 同位变换）。
5. **不跨协议比指标**。被问延迟/精度，先报口径（帧数、平台、是否含预处理）再报数。

---

2. 四个可复用的答题故事（STAR，面试官会反复挖）

华为技术面最爱链路化追问："为什么这么设计→试过哪些替代→怎么证明是它起作用→失败怎么定位"。这四段准备好，能覆盖 80% 的项目提问。

1. **跨模态对齐**：两模态分辨率/密度不一致 → 统一到 0.6m BEV 网格；对比 concat / SE 门控 / CBAM 的差异；用逐类 fn/fp 成分解剖证明增益不是超参噪声。
2. **训练稳定性三连**：CE target 越界（cls_num 参数名坑）→ CUBLAS_STATUS_ALLOC_FAILED（3→8 GEMM workspace，改 depthwise+pool）→ GT-Paste spconv 越界。**讲"怎么定位"比讲"修了什么"值分**：看日志→二分变量→最小复现。
3. **数据/评测的口径 bug**：转换器两个方向相反的 filter 版本 → mask 体积对不上（约 0.64 倍）→ 用逐元素对比 + IoU=1.0 锁定版本 → 结论是"掉分是转换器 bug 不是类别树变差"。这是**极强的工程严谨性证明**。
4. **端侧部署**：NPU 不支持算子 → 解耦 + 向量化 → 误检下降。升华句："端侧不是把模型塞进去，是把不可部署的语义换成可部署的等价语义。"

---

3. 手撕题（核心）——机器学习部分

> 面试手撕的机器学习和力扣的区别：**力扣考正确性，机器学习手撕考"你知不知道工业实现的坑"**。 写完代码必须主动补 2 句追问点，这是加分位。

3.1 Softmax（+ 数值稳定 + 交叉熵）

```python
import numpy as np

def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)      # 防 exp 溢出，数学上恒等
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)

def log_softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    return x - np.log(np.exp(x).sum(axis=axis, keepdims=True))
```

PyTorch 版交叉熵（**高频手撕**）：

```python
import torch
def cross_entropy(logits, target, weight=None):      # logits:[N,C] target:[N] int
    m = logits.max(dim=1, keepdim=True).values
    lse = m.squeeze(1) + torch.log(torch.exp(logits - m).sum(dim=1))
    nll = lse - logits.gather(1, target[:, None]).squeeze(1)
    if weight is not None:
        w = weight[target]
        return (nll * w).sum() / w.sum()
    return nll.mean()
```

追问必答： - 为什么要减 max？exp 上溢成 inf，再除 inf 得 nan；减常数不改变结果。 - log_softmax 比 softmax 稳在哪？交叉熵直接在对数域算，避免 "先算小概率→取 log" 的下溢；用 logsumexp 恒等式。 - softmax 温度 T 的作用：T↑ 分布更平（蒸馏/采样多样性），T↓ 更尖（argmax 近似）。 - 带 mask 的 softmax：被屏蔽位置置 **-1e9 或 -inf**，注意**整行全屏蔽会出 nan**，工业实现会 `scores.masked_fill(mask, -1e4)` 或对全屏蔽行补 0。 - Softmax vs Sigmoid：多分类互斥 vs 多标签独立（BCEWithLogits 更稳，因为它自带 logsumexp 形式）。 - Label Smoothing 怎么改？target 从 one-hot 变 (1-ε) 的 one-hot + ε/C 均匀。

3.2 K-Means

```python
import numpy as np

def kmeans(X, k, iters=100, tol=1e-4, seed=0, n_init=1):
    X = np.asarray(X, dtype=np.float64)
    rng = np.random.default_rng(seed)
    best_labels, best_centers, best_inertia = None, None, np.inf
    for _ in range(n_init):
        idx = rng.choice(len(X), k, replace=False)        # 简单随机初始化
```

```
        centers = X[idx].copy()
        labels = np.zeros(len(X), dtype=int)
        for _ in range(iters):
            d = ((X[:, None, :] - centers[None, :, :]) ** 2).sum(-1)   # n*k
            new_labels = d.argmin(1)
            new_centers = centers.copy()
            for j in range(k):
                m = new_labels == j
                if m.any():                                  # 空簇：保留旧中心
                    new_centers[j] = X[m].mean(0)
            shift = np.linalg.norm(new_centers - centers)
            centers, labels = new_centers, new_labels
            if shift < tol:
                break
        inertia = ((X - centers[labels]) ** 2).sum()
        if inertia < best_inertia:
            best_labels, best_centers, best_inertia = labels, centers, inertia
    return best_labels, best_centers
```

追问必答： - 复杂度 O(n · k · d · iters)，k-means++ 初始化把随机初始化的最坏情况压掉。 - k 怎么定：肘部法（inertia 拐点）、轮廓系数 silhouette、Gap statistic、业务约束。 - 假设：簇是**凸的、各向同性、方差相近**；对异常值敏感（可用 k-medoids / trimmed）。 - 与 EM/GMM：k-means 是 GMM 的硬分配特例（等协方差），EM 的 E 步=软分配。 - 空簇处理：保留旧中心 / 取离其他中心最远点。 - 加速：MiniBatch、kd-tree/球树（低维）、三角不等式剪枝（Elkan）。 - 面试官可能顺手问 DBSCAN：密度可达、不需 k、能找任意形状、对 eps/minPts 敏感、高维失效。

3.3 Self-Attention / Multi-Head Attention（最可能被要求写的一个）

先写 Self-Attention（单头），再扩多头，最后加 mask。用 PyTorch：

```
import math, torch, torch.nn as nn, torch.nn.functional as F

def self_attention(x, Wq, Wk, Wv, mask=None):
    # x: [B, L, D]; Wq/Wk/Wv: [D, D]
    Q, K, V = x @ Wq, x @ Wk, x @ Wv                  # [B, L, D]
    scores = Q @ K.transpose(-2, -1) / math.sqrt(Q.size(-1))   # [B, L, L]
    if mask is not None:
        scores = scores.masked_fill(mask, float('-inf'))       # mask=True 表示屏蔽
    attn = torch.softmax(scores, dim=-1)
    return attn @ V, attn
```

多头（完整、含 mask、可跑）：

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_head, dropout=0.0):
        super().__init__()
        assert d_model % n_head == 0
        self.h = n_head
        self.d_k = d_model // n_head
        self.w_q = nn.Linear(d_model, d_model)
        self.w_k = nn.Linear(d_model, d_model)
        self.w_v = nn.Linear(d_model, d_model)
        self.w_o = nn.Linear(d_model, d_model)
        self.drop = nn.Dropout(dropout)

    def forward(self, q, k, v, attn_mask=None, key_padding_mask=None):
        B, Lq, _ = q.shape
        Lk = k.size(1)
        # 1) 线性投影 + 拆头: [B,L,D] -> [B,h,L,dk]
        Q = self.w_q(q).view(B, Lq, self.h, self.d_k).transpose(1, 2)
        K = self.w_k(k).view(B, Lk, self.h, self.d_k).transpose(1, 2)
        V = self.w_v(v).view(B, Lk, self.h, self.d_k).transpose(1, 2)
        # 2) 缩放点积
        scores = Q @ K.transpose(-2, -1) / math.sqrt(self.d_k)    # [B,h,Lq,Lk]
        # 3) 两种 mask: 因果/结构 mask [Lq,Lk]; padding mask [B,Lk]
        if attn_mask is not None:
            scores = scores.masked_fill(attn_mask.bool(), float('-inf'))
        if key_padding_mask is not None:
            pad = key_padding_mask[:, None, None, :].bool()       # [B,1,1,Lk]
            scores = scores.masked_fill(pad, float('-inf'))
        # 4) softmax 后合并
        attn = self.drop(torch.softmax(scores, dim=-1))
        out = attn @ V                                            # [B,h,Lq,dk]
```

```
        out = out.transpose(1, 2).contiguous().view(B, Lq, self.h * self.d_k)
        return self.w_o(out), attn
```

Numpy 版（若面试官说"别用框架"）：

```
def mha_numpy(x, Wq, Wk, Wv, Wo, h, mask=None):
    B, L, D = x.shape
    dk = D // h
    def split(t):                                  # [B,L,D] -> [B,h,L,dk]
        return t.reshape(B, L, h, dk).transpose(0, 2, 1, 3)
    Q, K, V = split(x @ Wq), split(x @ Wk), split(x @ Wv)
    scores = Q @ K.transpose(0, 1, 3, 2) / np.sqrt(dk)          # [B,h,L,L]
    if mask is not None:
        scores = np.where(mask, -1e9, scores)
    scores = scores - scores.max(-1, keepdims=True)
    e = np.exp(scores)
    attn = e / e.sum(-1, keepdims=True)
    out = (attn @ V).transpose(0, 2, 1, 3).reshape(B, L, D)
    return out @ Wo, attn
```

追问必答（**这部分比代码更值分**）： - **为什么除 $\sqrt{d_k}$**：q·k 的方差随 d_k 线性增长，不缩放会让 softmax 落到饱和区、梯度消失；缩放使方差归一。 - **复杂度**：时间/显存 $O(L^2 \cdot d)$。这就是长上下文贵的根因 → 引出去向：FlashAttention（分块 + 在线 softmax，**省显存不省 FLOPs**）、MQA/GQA（共享 K/V 头，省 KV cache）、稀疏/滑窗注意力、线性注意力、Mamba/SSM。 - **$O(L^2)$ 显存怎么省**：FlashAttention 的 tiling + 重计算（不落 L×L 矩阵）、KV cache 量化、GQA。 - **view 后为什么必须 contiguous**：transpose 后内存不连续，view 要求连续，否则报错——实际实现写 `reshape` 或加 `.contiguous()`。 - **mask 语义一定要说清**：True=屏蔽（-inf）、False=保留；padding mask 是 batch 维、causal mask 是 Lq×Lk 下三角。 - **为什么要多头**：不同头学不同子空间/不同依赖模式（位置、句法、指代），可类比 CNN 多通道。 - **LayerNorm 放哪**：Post-LN（原始 Transformer，需 warmup）vs Pre-LN（现代主流，更稳，可去 warmup）；RMSNorm 少了均值中心化，更快。 - **Q/K/V 不同源**：self-attention（同源）vs cross-attention（Q 来自 decoder/query，K/V 来自 encoder，就是 Flamingo/BLIP-2/多模态融合里的融合入口——**这里一定要接上你的经验**）。 - 位置编码：绝对正弦、可学习、RoPE（相对位置、可外推、LLaMA 系）、ALiBi。**RoPE 为什么能外推/怎么扩 context（NTK 插值/位置内插）** 是当前高频追问。

3.4 分类树（CART，ID3/C4.5 关系要会讲）

```
import numpy as np

class Node:
    __slots__ = ("feat", "thr", "left", "right", "value")
    def __init__(self): self.feat = self.thr = self.left = self.right = self.value = None

def _gini(y, n_cls):
    if len(y) == 0: return 0.0
    p = np.bincount(y, minlength=n_cls) / len(y)
    return 1.0 - (p ** 2).sum()

def _build(X, y, depth, max_depth, min_samples_leaf, min_gain, n_cls):
    node = Node()
    node.value = np.bincount(y, minlength=n_cls).argmax()
    # 停止条件：深度、纯节点、样本太少
    if depth >= max_depth or len(np.unique(y)) == 1 or len(y) < 2 * min_samples_leaf:
        return node
    best_gain, best = min_gain, None
    parent = _gini(y, n_cls)
    n = len(y)
    for f in range(X.shape[1]):
        vals = np.unique(X[:, f])
        thrs = (vals[:-1] + vals[1:]) / 2.0
        for t in thrs:
            m = X[:, f] <= t
            nl, nr = m.sum(), n - m.sum()
            if nl < min_samples_leaf or nr < min_samples_leaf:
                continue
            g = parent - (nl * _gini(y[m], n_cls) + nr * _gini(y[~m], n_cls)) / n
            if g > best_gain:
                best_gain, best = g, (f, t, m)
    if best is None:
        return node
    f, t, m = best
    node.feat, node.thr = f, t
    node.left  = _build(X[m],  y[m],  depth + 1, max_depth, min_samples_leaf, min_gain, n_cls)
```

```
    node.right = _build(X[~m], y[~m], depth + 1, max_depth, min_samples_leaf, min_gain, n_cls)
    return node

def _predict_one(node, x):
    while node.feat is not None:
        node = node.left if x[node.feat] <= node.thr else node.right
    return node.value

class DecisionTreeClassifier:
    def __init__(self, max_depth=5, min_samples_leaf=1, min_gain=1e-7):
        self.max_depth, self.min_samples_leaf, self.min_gain = max_depth, min_samples_leaf, min_gain
    def fit(self, X, y):
        self.n_cls = int(y.max()) + 1
        self.root = _build(np.asarray(X, float), np.asarray(y, int), 0,
                           self.max_depth, self.min_samples_leaf, self.min_gain, self.n_cls)
        return self
    def predict(self, X):
        return np.array([_predict_one(self.root, x) for x in np.asarray(X, float)])
```

追问必答： - **划分准则**：ID3 信息增益（偏多值特征，倾向多叉）、C4.5 信息增益率（除法惩罚）、CART 基尼指数（**二叉树**，分类用基尼、回归用 MSE，叶子取均值）。 - **为什么 CART 用基尼不用熵**：二次形式没有 log，快；两者结论高度一致。 - **剪枝**：预剪枝（max_depth/min_samples_leaf/min_gain，快但可能欠拟合，贪心视野短）vs 后剪枝（代价复杂度 CART 的 α，从全树自底向上合并，更准但贵）。 - **过拟合原因**：划分太细，叶子只剩 1 个样本 → 方差爆炸。控制树深 / 叶最小样本 / 后剪枝。 - **连续特征处理**：排序后取中点枚举 O(n · log n · d) 的排序 + O(n · d) 扫描；大样本用分位分桶近似（LightGBM 直方图）。 - **特征重要性**：按该特征分裂带来的加权不纯度下降累积（sklearn 的 gini_importance），偏多值特征；更可靠是置换重要性。 - **推导到 GBDT**：残差/梯度拟合、XGBoost 的二阶 Taylor + 正则（$\gamma T + \lambda w^2/2$）+ 并行建直方图、LightGBM 的 leaf-wise + GOSS + EFB。 - 顺带：随机森林 = bagging + 特征子采样，降方差；树模型不需要归一化、对单调变换不变、不能外推。

3.5 机器学习高频补充（能写就写，5 分钟级）

```
# IoU
def iou(box_a, box_b):  # [x1,y1,x2,y2]
    ix1, iy1 = max(box_a[0], box_b[0]), max(box_a[1], box_b[1])
    ix2, iy2 = min(box_a[2], box_b[2]), min(box_a[3], box_b[3])
    iw, ih = max(0., ix2 - ix1), max(0., iy2 - iy1)
    inter = iw * ih
    area_a = max(0., box_a[2]-box_a[0]) * max(0., box_a[3]-box_a[1])
    area_b = max(0., box_b[2]-box_b[0]) * max(0., box_b[3]-box_b[1])
    union = area_a + area_b - inter
    return inter / union if union > 0 else 0.

# NMS
def nms(boxes, scores, thr=0.5):
    order = scores.argsort()[::-1]
    keep = []
    while order.size > 0:
        i = order[0]; keep.append(i)
        if order.size == 1: break
        ious = np.array([iou(boxes[i], boxes[j]) for j in order[1:]])
        order = order[1:][ious <= thr]
    return keep

# LayerNorm（手写反向是进阶题）
def layer_norm(x, gamma, beta, eps=1e-5):
    mu = x.mean(-1, keepdims=True)
    var = x.var(-1, keepdims=True)
    return gamma * (x - mu) / np.sqrt(var + eps) + beta

# BatchNorm 训练/推理差异 + 动量更新
# BN: 对 (N,H,W) 求统计量，依赖 batch（大 batch 才稳，小 batch 用 GroupNorm/LN）
# LN: 对单样本所有通道求，与 batch 无关，RNN/Transformer 标配
```

必背的还有：AUC 手算（按 score 排序后累加正样本 rank）、Top-1/Top-5、mAP、F1/Precision/Recall 与阈值、ROC vs PR 不平衡数据下的选择、L1/L2 为什么 L1 稀疏（等高线交点落在轴上/次梯度）、Dropout 训练-推理的缩放、Adam 的一阶二阶矩偏差修正、学习率 warmup+cosine 为什么有效、类别不平衡（focal loss、重采样、代价敏感）、梯度消失/爆炸（初始化、残差、LN、梯度裁剪）、双 seed 噪声地板（**这条你有实战，主动讲**）。

4. 手撕题——力扣部分（华为风格）

4.1 先明确考试形态

- **校招机试**：3 题 / 400 分 / 120 分钟，难度约 简单100 + 中等200 + 困难300，**ACM 模式**（自己读 stdin、自己 print），支持本地 IDE 调试。多数组只做出 200 分就能进面，**但第一题必须稳**。
- **技术面手撕**：面试官现场给题，共享屏幕写，通常 LeetCode 中等，要求能跑通 + 说复杂度。**边写边说出思路、先讲暴力再优化**，比默写满分解更受认可。
- 华为偏好：**字符串处理、模拟、DFS/BFS、DP、二叉树、数组双指针**。硬核图论/线段树极少。

4.2 ACM 模式 IO 模板（务必背，很多人挂在这）

```python
import sys

def main():
    data = sys.stdin.read().split()      # 万能：全部按空白切分
    if not data: return
    # 例：第一行 n，之后 n 个整数
    n = int(data[0])
    nums = list(map(int, data[1:1 + n]))
    print(sum(nums))

if __name__ == "__main__":
    main()
```

```python
# 多行/多组的稳健写法
import sys
lines = [l.strip() for l in sys.stdin if l.strip()]
for line in lines:
    a, b = map(int, line.split())
    print(a + b)
```

注意：华为题常有多组输入直到 EOF、以及行内用逗号/空格混排；**读进来先 split，不要相信行数**。

4.3 必刷清单（按优先级，20 + 15）

P0（闭眼能写，两天刷完）

#	题	考点
1	两数之和 / 三数之和	哈希、排序+双指针
2	无重复字符的最长子串	滑动窗口
3	最长回文子串	中心扩展（DP 次选）
4	有效括号	栈
5	字符串反转 / 翻转单词 / 大数相加	字符串模拟
6	合并两个有序链表 / 反转链表	链表指针
7	环形链表 / 相交链表	快慢指针
8	LRU 缓存	哈希 + 双向链表（**华为高频**）
9	二叉树层序遍历 / 最大深度 / 之字形	BFS/DFS
10	验证二叉搜索树	中序 or 上下界
11	岛屿数量 / 岛屿最大面积	DFS 淹没
12	腐烂的橘子	多源 BFS
13	全排列 / 子集 / 组合总和	回溯模板
14	爬楼梯 / 打家劫舍	线性 DP
15	编辑距离 / 最长公共子序列	二维 DP

#	题	考点
16	最大子数组和	贪心/DP
17	数组中第 K 大 / 前 K 高频	堆 or 快速选择
18	搜索旋转排序数组	二分变体
19	零钱兑换	完全背包
20	手写快排 / 归并 / 堆排	排序（面试常直接考）

P1（有余力）：接雨水、滑动窗口最大值（单调队列）、最小覆盖子串、最长递增子序列（O(nlogn) 贪心+二分）、01 背包、课程表（拓扑）、朋友圈（并查集）、二叉树最近公共祖先、路径总和 II、字符串解码、螺旋矩阵、复原 IP 地址。

4.4 五个模板（现场默写用）

```python
# 1) 快排（含随机化，避免有序数组退化）
import random
def quick_sort(a, l=0, r=None):
    if r is None: r = len(a) - 1
    if l >= r: return a
    p = random.randint(l, r)
    a[l], a[p] = a[p], a[l]
    pivot, i = a[l], l
    for j in range(l + 1, r + 1):
        if a[j] < pivot:
            i += 1
            a[i], a[j] = a[j], a[i]
    a[l], a[i] = a[i], a[l]
    quick_sort(a, l, i - 1); quick_sort(a, i + 1, r)
    return a

# 2) 二分（左边界，防死循环）
def lower_bound(a, x):
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] < x: lo = mid + 1
        else: hi = mid
    return lo

# 3) 滑动窗口（无重复最长子串）
def length_of_longest(s):
    last, left, ans = {}, 0, 0
    for i, c in enumerate(s):
        if c in last and last[c] >= left:
            left = last[c] + 1
        last[c] = i
        ans = max(ans, i - left + 1)
    return ans

# 4) 小顶堆 TopK（n log k）
import heapq
def topk(nums, k):
    h = []
    for x in nums:
        heapq.heappush(h, x)
        if len(h) > k: heapq.heappop(h)
    return h

# 5) 并查集
class DSU:
    def __init__(self, n): self.p = list(range(n)); self.r = [0]*n
    def find(self, x):
        while self.p[x] != x:
            self.p[x] = self.p[self.p[x]]
            x = self.p[x]
        return x
    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb: return False
        if self.r[ra] < self.r[rb]: ra, rb = rb, ra
        self.p[rb] = ra
```

```
        if self.r[ra] == self.r[rb]: self.r[ra] += 1
        return True
```

写题时的三个习惯：**先说复杂度 → 写边界（空、单元素、全相同、溢出）→ 写完自己过一遍样例**。

5. 多模态八股速记（12 问，各 30 秒版）

> 这里是**口头速记版**；完整答案库（118 问，按面试顺序 + 追问 + 项目接口话术 + 融合调控/损失函数专章）见同目录《华为-多模态大模型-八股知识问答_20261029.docx/md/pdf》。

1. **CLIP 怎么训的**：图像/文本双塔，batch 内 N×N 相似度矩阵，对角为正样本，对称 InfoNCE（in-batch negatives）。温度 τ 可学习。零样本靠 prompt + 文本分类器权重。
2. **CLIP 的局限**：只做粗粒度全局对齐，不擅长计数/空间关系/细粒度；对比学习不考虑词序（bag-of-words 问题）。
3. **BLIP-2 的 Q-Former**：冻结图像塔 + 冻结 LLM，中间用固定数量 learnable query 通过 cross-attention 抽取视觉信息，把可变长 patch 压成固定 token（省 LLM 上下文）。两阶段：表征学习（ITC/ITM/ITG）→ 生成学习（接 LLM 做 LM loss）。
4. **LLaVA 的 projector**：两层 MLP 直接映射视觉 token，简单有效（消融显示比 Q-Former 不差）；两阶段训练=先冻结 LLM 只训 projector（对齐），再解冻部分/全量做指令微调。
5. **ViT vs CNN**：patch 化 + 全局注意力、无归纳偏置、需要大数据或强增广/蒸馏；计算 $O(N^2)$，高分辨率要用窗口/分层（Swin）。
6. **视觉 token 太多怎么办**：Q-Former/Perceiver 压缩、池化、token merge、动态分辨率、只取关键帧。**接你的经验：BEV query 重排与深度加权投影就是"几何感知的 token 组织"**。
7. **位置编码**：正弦/可学习/RoPE/ALiBi；RoPE 保相对位置、LLaMA 系标准；长上下文靠位置内插/NTK/缩放。
8. **训练三阶段**：预训练（对齐）→ SFT（指令遵循）→ 偏好对齐（RLHF/DPO）。DPO 不用 reward model，直接用偏好对做分类式损失。
9. **LoRA**：低秩分解 ΔW=BA，只训 A/B，rank 8~64、alpha 缩放、可合并推理零开销。
10. **幻觉**：视觉 token 与语言先验冲突时模型倾向语言先验（POPE 评测）；缓解=更强视觉监督、负样本、解码策略、RAG。
11. **评测**：MMBench/MMMU/SEED（综合），POPE（幻觉），RefCOCO/Visual Grounding，VideoMME（长视频）。说清"评什么、怎么算分"就够。
12. **端侧/昇腾**：CANN、ATC（ONNX→om）、AOE 调优、量化 INT8/INT4、算子不支持要替换。**你有 ATC/MDC610 真机经验，这题是送分题，务必主动引到这儿**。

6. 3 天冲刺表

时间	任务
D1 上午	改简历（1.1 映射表逐条过）+ 背开场 30 秒自述 + 红线
D1 下午	手撕 softmax/CE、kmeans、self-attention 单头 → 多头 mask 版（手写 3 遍不看书）
D1 晚	力扣 P0 前 10 题（限时 20 分钟/题，ACM 模式输入输出）
D2 上午	手撕 CART + 追问点复述（划分准则/剪枝/GBDT 推导）
D2 下午	力扣 P0 后 10 题 + 五个模板默写
D2 晚	多模态八股 12 问（能开口讲，不追求逐字）
D3 上午	四个 STAR 故事各讲 3 分钟并录音自听；补 CLIP/BLIP-2/LLaVA 结构图能画出来
D3 下午	全真模拟：一道力扣中等（边说边写）+ 一道机器学习手撕 + 项目深挖 15 分钟

7. 反问环节（3 个，显示你在想落地）

1. 团队的多模态模型主要在哪个模态组合上做（图-文 / 视频 / 点云-图），当前瓶颈在数据、结构还是推理成本?
2. 端侧和云侧的比例大概多少？模型要到什么时延/算力预算，昇腾侧是 ATC 自研链路还是 MindSpore 栈?
3. 新人前三个月最可能被派到哪一块（数据管线 / 训练 / 评测 / 部署）？

附：代码自测状态

第 3、4 章代码已落到可运行文件（与本文同一目录）：

- `华为多模态手撕题库_自测通过_20261029.py`

已跑通的断言：softmax/log_softmax（含 1e3 级大数不溢出、平移不变）；交叉熵与 `F.cross_entropy` 数值一致（含类别权重）；kmeans 从三个高斯簇正确恢复到 0.5 半径内；CART 二分类/三分类训练集 ACC=1.0 且根节点阈值落在簇间（x0≤1.362）；IoU/NMS 的期望框保留顺序；LayerNorm 零均值单位方差；力扣五个模板（快排随机化 50 组随机数组、lower_bound、滑窗、TopK、DSU）全通过。

注意：本机没有 torch 环境，**3.3 的 torch 版 MHA 是用同构 numpy 实现验证逻辑的**（单头/多头/因果 mask/padding mask 形状与 softmax 归一化均已验证），但 torch 的 `view/transpose/masked_fill/gather` 具体调用建议你在自己的环境里跑一遍再上场默写。

本文档基于你已有项目（BEV 多任务融合、OCC 数据闭环、标定与端侧部署）撰写，所有数字均来自既有记录；不构成虚构经历。